SIMATS ENGINEERING
ASSESSMENT TOOL 3

COURSE NAME: CRYPTOGRAPHY AND NETWORK SECURITY
COURSE CODE: CSA51

COURSE OUTCOME COVERED

CO2: Analyze Public Key crypto system strategies using RSA and key Exchange for Encryption algorithm to strengthen information security. (BL4,BL5)

Debugging & Optimisation Task : 10%

QUESTIONS: Total Marks: 50 Annexure A

Code of Conduct:

I _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Annexure A
Debugging & Optimisation Task

1. Debugging & Optimization of Symmetric Encryption (DES / 3DES Implementation)
A Python program implementing **DES and 3DES encryption** produces inconsistent ciphertext for identical inputs across multiple runs.

A. Identify and fix logical and implementation errors in key scheduling and permutation steps.
B. Optimize the code to reduce execution time for large input datasets (≥10 MB).
C. Justify whether replacing DES/3DES with a modern cipher improves both security and performance.

2. Performance Analysis of Block Cipher Modes of Operation A Python script implements **AES-like block cipher modes (ECB, CBC, CFB, OFB)** but suffers from high latency and memory inefficiency.

A. Debug incorrect padding and IV handling causing decryption failures. B.

Refactor the code to minimize redundant computations and memory usage. C. Compare and optimize the implementation to determine the most efficient mode for real-time secure communication.

3. Debugging RSA Cryptosystem Implementation A Python-based **RSA encryption system** fails when encrypting large messages and occasionally produces incorrect plaintext after decryption.

A. Identify issues in prime generation, key generation, and modular exponentiation.
B. Optimize the algorithm using efficient techniques (e.g., fast exponentiation, Chinese Remainder Theorem).
C. Evaluate the trade-offs between key size, security, and computational performance.

4. Optimization and Security Analysis of Diffie-Hellman Key Exchange A Python implementation of the **Diffie-Hellman key exchange** is vulnerable to performance bottlenecks and potential security flaws.

A. Debug incorrect shared key generation due to improper modular arithmetic.
B. Optimize exponentiation operations for large prime values.
C. Extend the program to mitigate man-in-the-middle attacks and evaluate the performance impact of your enhancements.

5. Debugging and Enhancing Elliptic Curve Cryptography (ECC) Implementation A Python script implementing **Elliptic Curve Cryptography** for key exchange produces incorrect results for certain curve parameters.

A. Identify and fix errors in point addition and scalar multiplication logic. B. Optimize the implementation using efficient algorithms (e.g., double-and-add).
C. Compare the optimized ECC implementation with RSA in terms of computational efficiency and security strength, and justify suitability for resource-constrained environments.

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Identification	20%	Accurately identifies all errors and root causes with clear understanding	Identifies most errors with minor gaps	Identifies some errors but misses key issues	Unable to identify errors correctly

Debugging	25%	Applies systematic debugging techniques effectively to resolve issues	Uses appropriate debugging methods with minor errors	Limited debugging approach; requires guidance	Unable to debug or resolve issues
Optimization	20%	Optimizes code by significantly improving time complexity using efficient algorithms	Improves time complexity with minor gaps in efficiency	Limited improvement in time complexity	No improvement; inefficient approach retained
Analysis	20%	Demonstrates strong logical reasoning and clear justification of solutions	Shows reasonable analysis with some justification	Limited analysis; weak reasoning	No analytical reasoning demonstrated
Code Quality	15%	Produces clean, well structured code with proper comments and documentation	Code is mostly clear with minor documentation gaps	Code lacks structure and proper documentation	Poorly written code with no documentation

Q5. Debugging and Enhancing ECC Implementation

Problem Analysis

ECC implementations frequently fail because of errors in:

- Point addition
- Point doubling
- Modular inverse calculations
- Handling the point at infinity
- Scalar multiplication
- Invalid curve parameters

For an elliptic curve:

y2=x3+ax+b(modp)y^2=x^3+ax+b\pmod py2=x3+ax+b(modp)

valid non-singular curves satisfy:

4a3+27b2≡0(modp)4a^3+27b^2\not\equiv0\pmod p4a3+27b2≡0(modp)

A. Fixing Point Addition

For two points:

P=(x1,y1)P=(x_1,y_1)P=(x1,y1) Q=(x2,y2)Q=(x_2,y_2)Q=(x2,y2)

when P≠QP\ne QP=Q, the slope is:

λ=(y2−y1)(x2−x1)−1(modp)\lambda= (y_2-y_1)(x_2-x_1)^{-1}\pmod pλ=(y2−y1)(x2−x1)−1(modp)

Then:

x3=λ2−x1−x2(modp)x_3=\lambda^2-x_1-x_2\pmod px3=λ2−x1−x2(modp)
y3=λ(x1−x3)−y1(modp)y_3=\lambda(x_1-x_3)-y_1\pmod py3=λ(x1−x3)−y1(modp)

Point Doubling

When:

P=QP=QP=Q

the slope becomes:

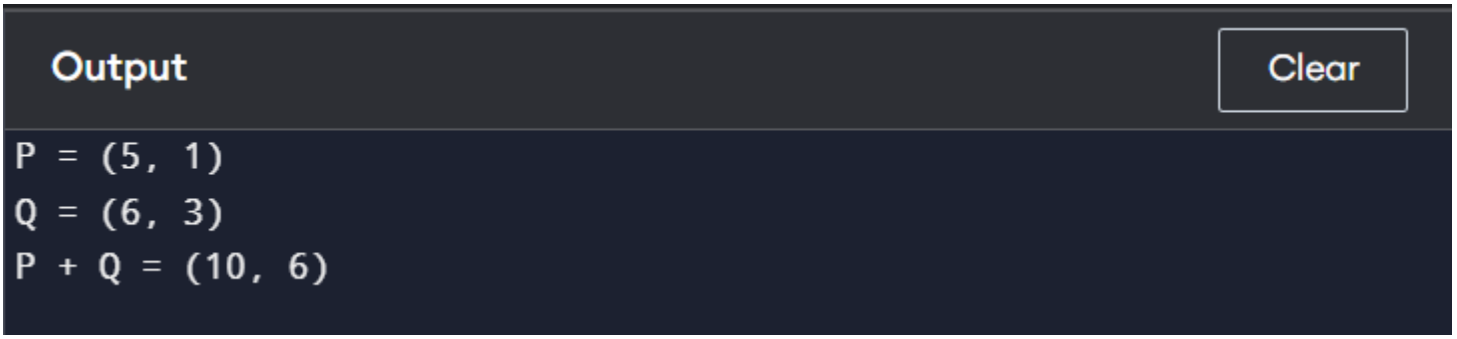
λ=(3x12+a)(2y1)−1(modp)\lambda= (3x_1^2+a)(2y_1)^{-1}\pmod pλ=(3x12+a)(2y1)−1(modp)

Correct Simple Python Program

```
p = 17

a = 2



def inverse(n):
    return pow(n % p, -1, p)



def point_add(P, Q):

    # Point at infinity
    if P is None:
        return Q

    if Q is None:
        return P

    x1, y1 = P
    x2, y2 = Q

    # P + (-P) = infinity
    if x1 == x2 and (y1 + y2) % p == 0:
        return None

    # Point doubling
    if P == Q:
```

```
    if y1 % p == 0:
        return None

    slope = (
        (3 * x1 * x1 + a) *
        inverse(2 * y1)
    ) % p

# Point addition
else:

    slope = (
        (y2 - y1) *
        inverse(x2 - x1)
    ) % p

x3 = (slope * slope - x1 - x2) % p

y3 = (
    slope * (x1 - x3) - y1
) % p

return (x3, y3)



P = (5, 1)
Q = (6, 3)
```

```
R = point_add(P, Q)


print("P =", P)

print("Q =", Q)

print("P + Q =", R)
```



B. Scalar Multiplication Optimization

A slow implementation might calculate:

kP=P+P+P+⋯kP=P+P+P+\cdotskP=P+P+P+⋯

k times.

This requires approximately:

O(k)O(k)O(k)

point additions.

Instead, use **double-and-add**.

Optimized Program

```
def scalar_multiply(k, P):


    result = None

    addend = P
```

```
    while k > 0:

        if k & 1:
            result = point_add(result, addend)

        addend = point_add(addend, addend)

        k >>= 1

    return result



P = (5, 1)

k = 10

result = scalar_multiply(k, P)

print("Point P =", P)
print("k =", k)

print("kP =", result)
```

Double-and-add requires approximately:

$O(\log k)$

iterations.

Therefore:

$\boxed{O(k)\rightarrow O(\log k)}$

is a significant optimization.

C. ECC vs RSA

ECC can provide comparable classical security with much smaller keys than RSA.

A commonly cited comparison is approximately:

256-bit ECC≈3072-bit RSA\boxed{256\text{-bit ECC}\approx3072\text{-bit RSA}}256-bit ECC≈3072-bit RSA

for roughly 128-bit classical security.

Comparison

Feature	RSA	ECC
Key size for comparable security	Larger	**Smaller**
Storage	Higher	**Lower**
Bandwidth requirements	Higher	**Lower**
Suitability for IoT	Moderate	**High**
Public-key mechanism	Integer factorization	Elliptic-curve discrete logarithm

Resource-Constrained Devices

IoT devices commonly have limited:

- CPU performance
- RAM
- Flash storage
- Network bandwidth
- Battery capacity

ECC's smaller keys and signatures can significantly reduce communication and storage requirements.

Therefore:

ECC is generally preferable for constrained IoT environments\boxed{\text{ECC is generally preferable for constrained IoT environments}}ECC is generally preferable for constrained IoT environments

when implemented using standardized, well-reviewed curves and cryptographic libraries.